

LAPORAN PRAKTIKUM
TUGAS MODUL 1 & MODUL 2 IOT II

Disusun dalam rangka memenuhi salah satu syarat kelengkapan praktikum

Mata Kuliah Teknologi IOT II



Di Susun Oleh:

NAMA : M. ROMZI

NPM : 25782015

KELAS : 3A

PRODI : TRI

DOSEN PENGAMPU :

Dr. Ir. Septafiansyah Dwi Putra, S. T., M.T.

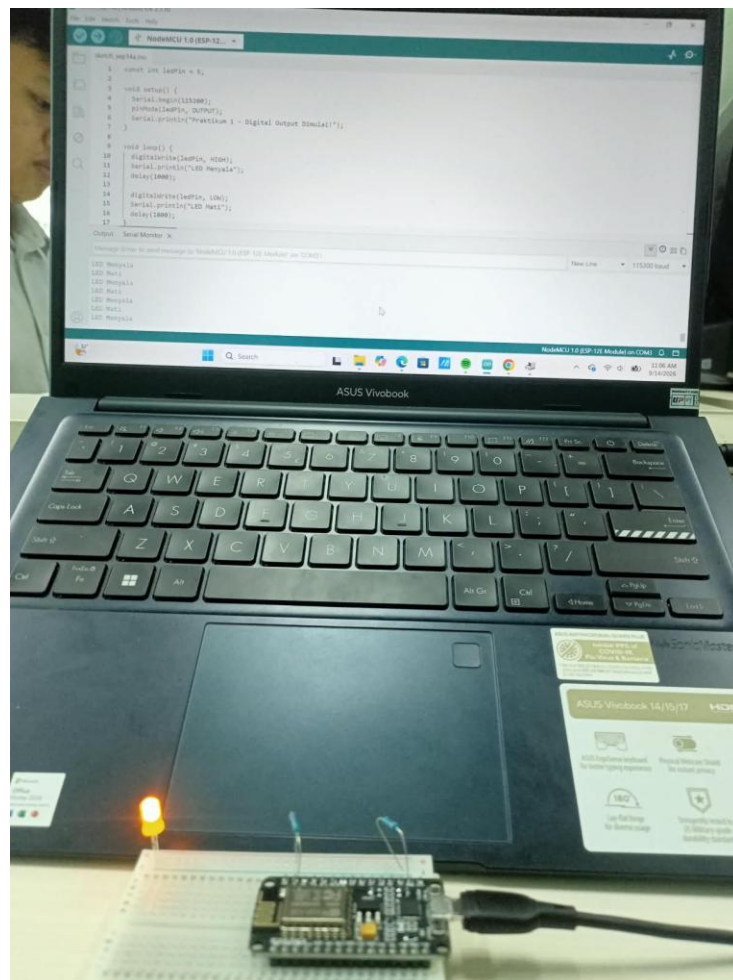
Agus Ambarwari, S.Pd.,M.Kom.

DINAS PENDIDIKAN DAN KEBUDYAAAN
KOTA BANDAR LAMPUNG
POLITEKNIK NEGERI LAMPUNG
TAHUN AJARAN 2026/2027

MODUL 1

Praktikum 1.5.1

Praktikum 1.5.1 ini bermaksud memprogram NodeMCU agar mengalirkan tegangan 3.3V (HIGH) secara terprogram untuk menyalakan komponen eksternal.

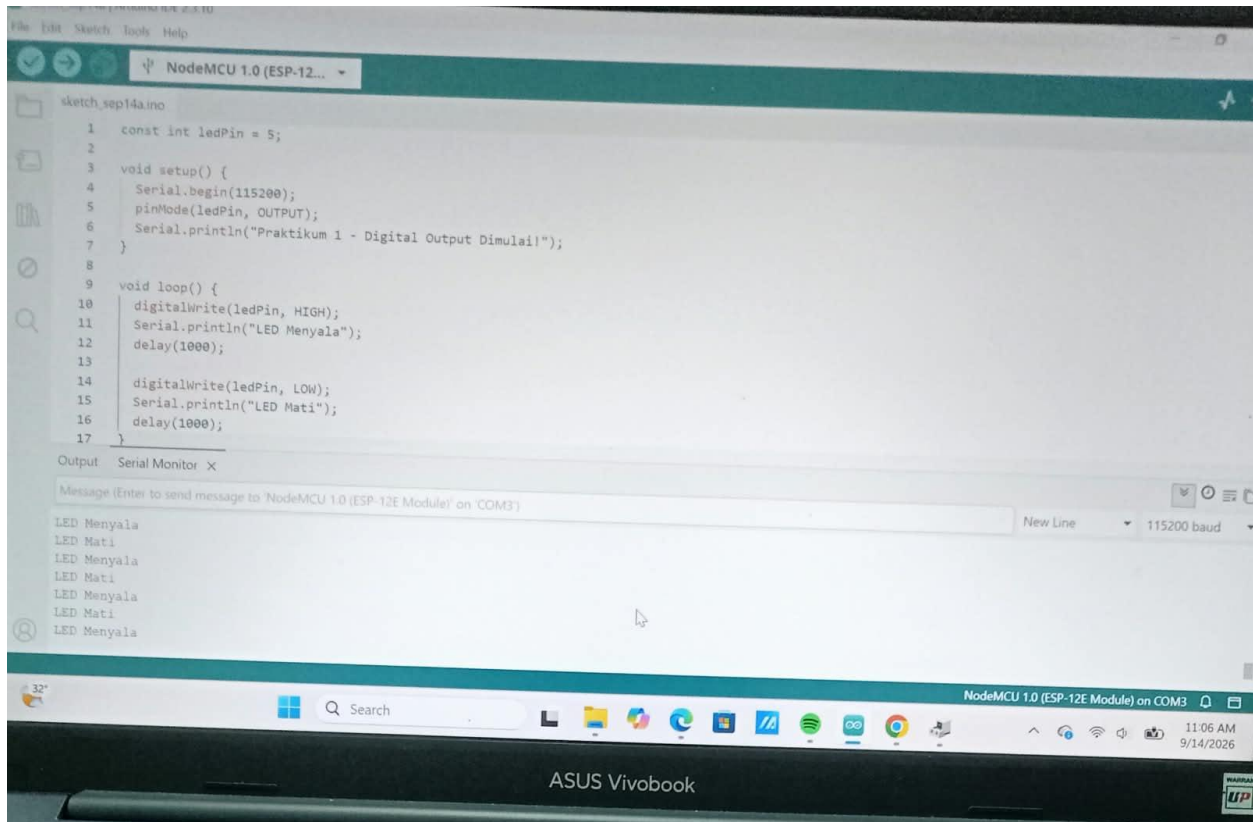


Langkah Kerja Rangkaian:

1. Pasang NodeMCU pada *breadboard*.
2. Pasang LED pada baris kosong di *breadboard*. Perhatikan kakinya:
 - Kaki yang **lebih panjang** adalah **Anoda (Positif)**.
 - Kaki yang **lebih pendek** (dan terdapat sisi datar di tepi plastik kepala LED) adalah **Katoda (Negatif)**.
3. Hubungkan pin **D1 (GPIO 5)** dari NodeMCU ke kaki **Anoda** LED menggunakan kabel *jumper*.

4. Hubungkan kaki **Katoda** LED ke salah satu ujung **Resistor 220 Ohm**.

5. Hubungkan ujung **Resistor** lainnya ke jalur **GND** pada NodeMCU. (Catatan: Resistor berfungsi sebagai pembatas arus agar LED tidak terbakar).



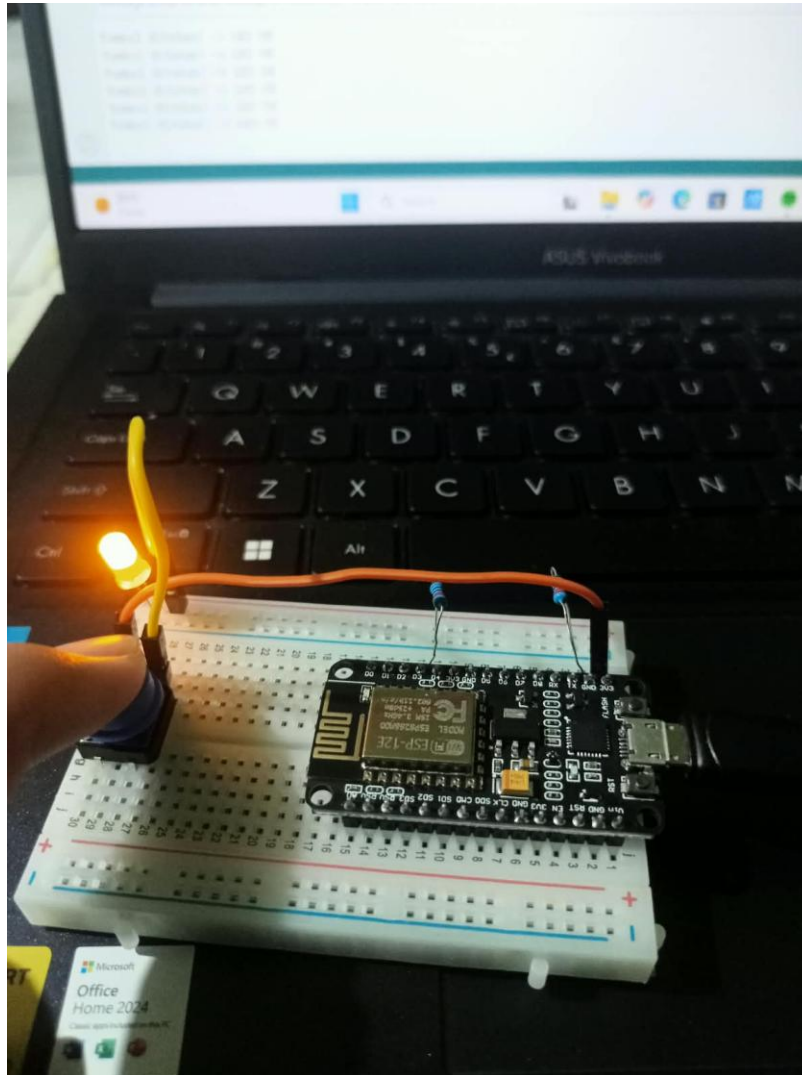
Gambar diatas merupakan code untuk menjalankan program

Penjelasan kode :

- **setup()**: Blok fungsi yang dieksekusi hanya **satu kali** saat NodeMCU dinyalakan atau di-*reset*. Biasanya digunakan untuk pengaturan awal.
- **loop()**: Blok fungsi yang dieksekusi secara **berulang-ulang** (terus menerus) selama NodeMCU memiliki daya.
- **pinMode(ledPin, OUTPUT)**: Memberi instruksi bahwa pin D1 (GPIO 5) akan digunakan sebagai jalur keluar (*Output*) tegangan.
- **digitalWrite(ledPin, HIGH)**: Memberikan tegangan 3.3V secara digital ke pin tersebut, sehingga arus mengalir dan LED menyala.
- **delay(1000)**: Menghentikan sementara eksekusi program (jeda) selama 1000 milidetik (1 detik).

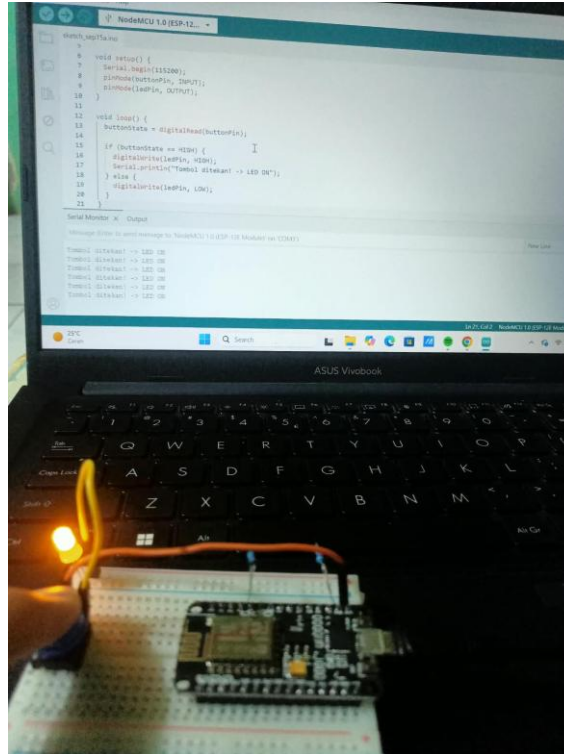
1.5.2 Praktikum 2: *Digital Input* (Membaca *Push Button*)

Praktikum ini bermaksud mendeteksi interaksi fisik dengan membaca status tegangan dari sebuah tombol, memanfaatkan rangkaian resistor *pull-down* eksternal.



Langkah Kerja Rangkaian:

1. Biarkan rangkaian LED pada pin D1 dari Praktikum 1 tetap terpasang.
2. Pasang sebuah *push button* pada *breadboard*, melintasi garis pemisah/parit tengah.
3. Hubungkan kaki pertama *push button* ke pin 3V3 (3.3V) pada NodeMCU.
4. Hubungkan kaki kedua *push button* ke pin D2 (GPIO 4) NodeMCU.
5. Pada kaki kedua yang sama, pasang sebuah Resistor 10k Ohm, lalu hubungkan ujung lain dari resistor tersebut ke jalur GND NodeMCU.



Gambar diatas merupakan code untuk menjalankan program

Penjelasan kode :

- `pinMode(buttonPin, INPUT)`: Menetapkan pin D2 (GPIO 4) sebagai jalur masuk (*Input*) untuk mendeteksi / membaca tegangan dari luar.
- `digitalRead(buttonPin)`: Fungsi yang bertugas membaca status tegangan pada pin saat ini. Hasilnya akan berupa HIGH (ada tegangan) atau LOW (tidak ada tegangan).
- `if (buttonState == HIGH)`: Blok logika *conditional*. Karena kita menggunakan rangkaian resistor *pull-down*, menekan tombol akan mengalirkan arus 3.3V ke pin mikrokontroler. Sehingga, status terbaca HIGH dan kondisi `if` terpenuhi (LED dinyalakan).

1.6 LATIHAN

- **Latihan 1 (Pemahaman Logika Pin)**: Berdasarkan tabel Referensi Pinout di subbab 1.3.4, jelaskan apa yang mungkin terjadi jika Anda menggunakan logika rangkaian dari Praktikum 2 (Active-High / *pull-down*), namun menyambungkan tombol tersebut ke pin **D8 (GPIO 15)**, lalu menekan tombol tersebut *tepat* pada detik di mana NodeMCU baru diberi daya?

Node mcu akan gagal booting atau sistemnya stuck mengapa, karena pin **D8 (GPIO 15)** punya aturan khusus yaitu *Pulled to GND*. Artinya, pin ini wajib dalam kondisi LOW pas *board* pertama kali dinyalakan. Kalau tombol ditekan pas proses *booting*, rangkaian *pull-down* justru ngirim sinyal HIGH (3.3V) ke D8, makanya proses nyala mikrokontroler-nya gagal.

- **Latihan 2 (Implementasi Kode):** Modifikasi kode pada Praktikum 1 agar LED berkedip secara *tidak simetris*, yaitu menyala sangat cepat selama 200 ms, kemudian mati agak lama selama 800 ms. Tuliskan dua baris fungsi `delay()` yang harus Anda ubah.

berikut kode yang di ubah :

```
void loop() {  
    digitalWrite(ledPin, HIGH);  
    Serial.println("LED Menyala");  
    delay(200); // Diubah dari 1000 ke 200 ms (menyala cepat)  
  
    digitalWrite(ledPin, LOW);  
    Serial.println("LED Mati");  
    delay(800); // Diubah dari 1000 ke 800 ms (mati lebih lama)  
}
```

```
9 void loop() {  
10     digitalWrite(ledPin, HIGH);  
11     Serial.println("LED Menyala");  
12     delay(200); // Diubah dari 1000 ke 200 ms (menyala cepat)  
13  
14     digitalWrite(ledPin, LOW);  
15     Serial.println("LED Mati");  
16     delay(800); // Diubah dari 1000 ke 800 ms (mati lebih lama)  
17 }
```

- **Latihan 3 (Analisis Sirkuit):** Berdasarkan hukum aliran arus listrik searah, apa yang terjadi jika pada Praktikum 1 Anda memasang kaki LED secara terbalik (Anoda ke GND, Katoda ke D1)? Apakah program akan mengalami *error* kompilasi? Mengapa LED tidak menyala?

Program tetap sukses di compile dan di upload, karena kesalahan pemasangan komponen fisik tidak memengaruhi struktur kode pemrograman. Led tidak menyala karena led memiliki polaritas (kutub positif dan negatif), dan hanya mengalirkan arus listrik satu arah. Jika dipasang terbalik, Led akan menahan aliran arus dari pin D1 ke GND, sehingga lampu tidak bisa menyala.

- **Latihan 4 (Pengembangan Algoritma):** Ubah blok kondisi if-else pada Praktikum 2 sehingga perilakunya terbalik: "Saat tombol TIDAK ditekan, LED **menyala**. Saat tombol DITEKAN, LED justru **mati**." :

```
void loop() {  
  buttonState = digitalRead(buttonPin);  
  
  if (buttonState == LOW) {  
    digitalWrite(ledPin, HIGH); // Pas tombol DILEPAS -> LED malah  
    NYALA  
  } else {  
    digitalWrite(ledPin, LOW); // Pas tombol DITEKAN -> LED malah  
    MATI  
  }  
}
```

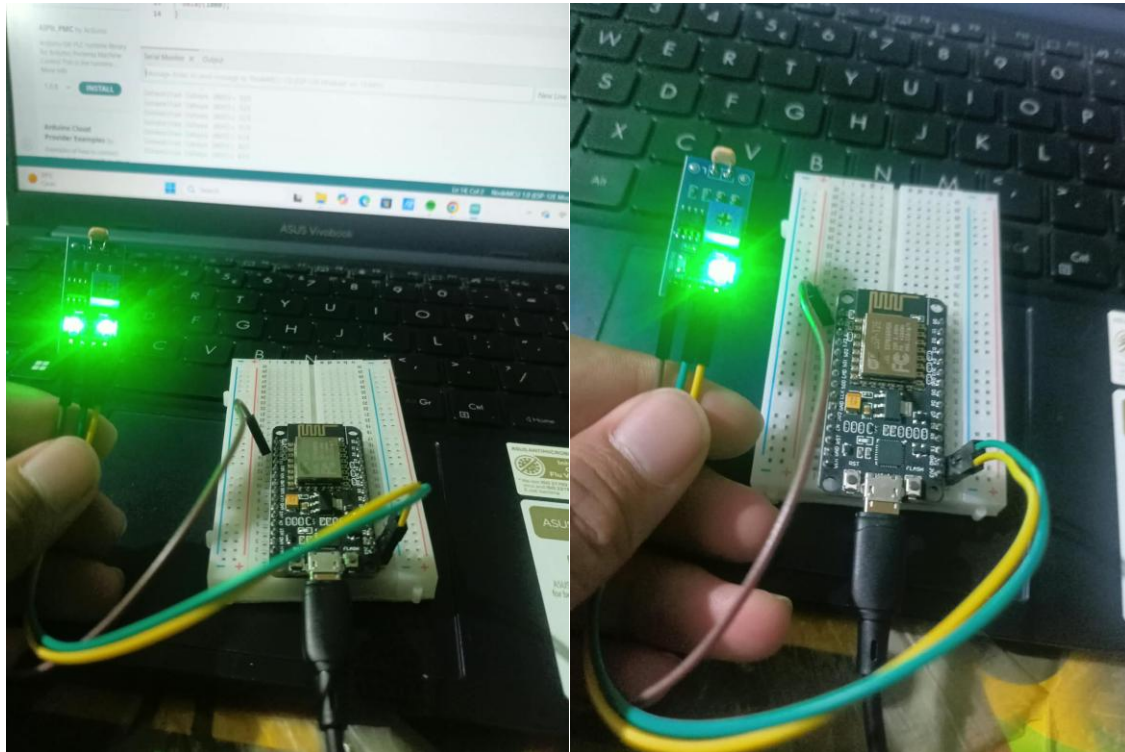
```
void loop() {  
  buttonState = digitalRead(buttonPin);  
  
  if (buttonState == LOW) {  
    digitalWrite(ledPin, HIGH); // Pas tombol DILEPAS -> LED malah NYALA  
  } else {  
    digitalWrite(ledPin, LOW); // Pas tombol DITEKAN -> LED malah MATI  
  }  
}
```

MODUL 2

2.5 Praktikum

2.5.1 Praktikum 1: Pembacaan Intensitas Cahaya (LDR) via ADC

Sensor LDR akan menurunkan hambatannya (resistansi) saat terpapar cahaya terang, dan meningkatkan resistansinya saat gelap. Untuk menerjemahkan perubahan resistansi ini menjadi variasi tegangan yang bisa diukur oleh pin A0, kita menggunakan sirkuit pembagi tegangan (*Voltage Divider*).



Langkah Kerja Rangkaian:

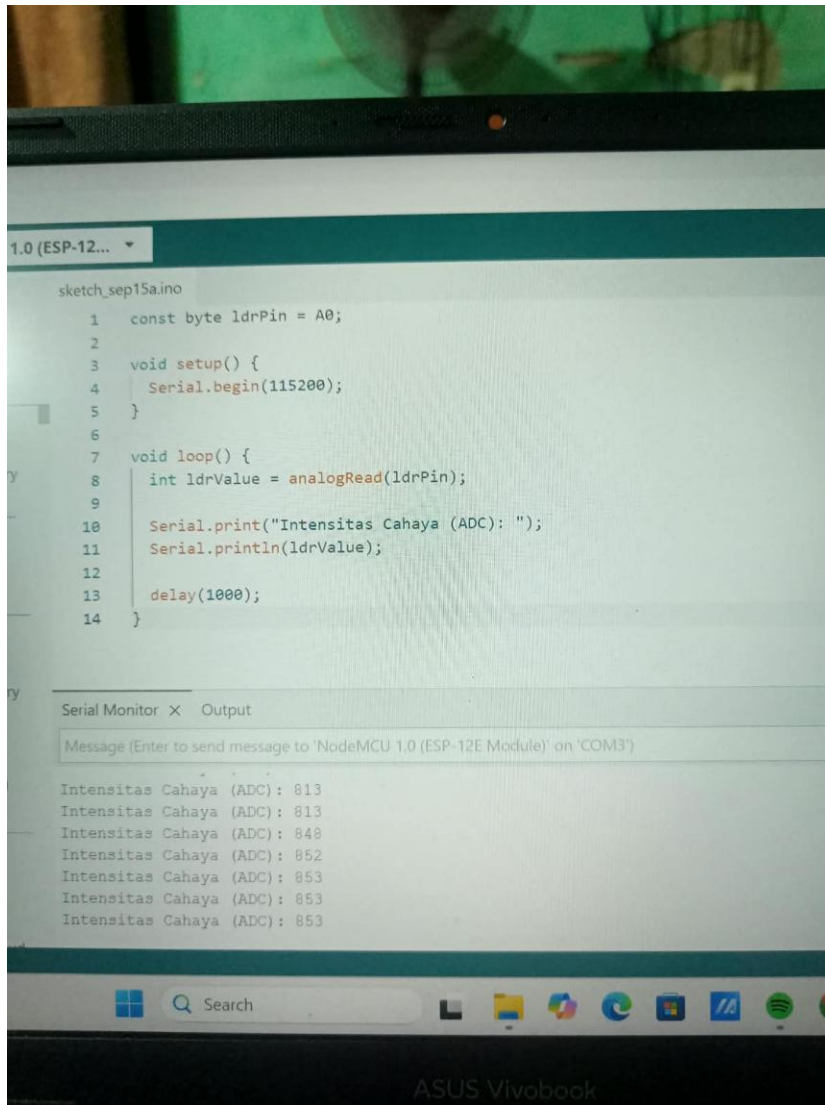
1. Tancapkan satu kaki LDR ke lubang *breadboard* yang terhubung langsung ke pin **3V3** (3.3 Volt) NodeMCU.
2. Tancapkan kaki LDR yang satunya ke sebuah jalur parit kosong di *breadboard*.
3. Hubungkan parit LDR tersebut dengan pin **A0** menggunakan sebuah kabel *jumper*.
4. Pada parit LDR yang sama (sejajar dengan kabel ke A0), pasang salah satu kaki **Resistor 10k Ohm**.
5. Hubungkan ujung hulu Resistor 10k Ohm tersebut ke rel **GND**.

(Logika Rangkaian: Saat cahaya terang, resistansi LDR kecil sehingga arus 3.3V membanjiri A0, menghasilkan nilai ADC besar mendekati 1023. Saat gelap, resistansi LDR sangat besar

menahan arus, sehingga pin A0 'tertarik' oleh resistor 10k ke arah GND, menghasilkan nilai ADC kecil mendekati 0).

Langkah Kerja Pemrograman:

1. Buat berkas baru (*New Sketch*).
2. Tuliskan kode program berikut:



3. Upload program. Buka **Serial Monitor** (pastikan setelan 115200 baud).
4. **Instruksi Pengujian:** Sorot LDR dengan lampu *flash* ponsel dan amati angka yang membesar. Lalu, tutup bagian atas LDR rapat-rapat dengan jari dan amati penurunannya.

Penjelasan Singkat Kode:

- `const byte ldrPin = A0;`; Mendeklarasikan pin ADC. Tipe `byte` digunakan (menggantikan `int`) untuk menghemat memori (*RAM*), karena ID pin tidak akan melebihi angka 255.
- `analogRead(ldrPin);`; Fungsi bawaan mikrokontroler untuk membaca level tegangan analog murni di pin A0 dan mengonversinya menjadi data integer 10-bit (berkisar antara 0 hingga 1023).

2.5.2 Praktikum 2: Membaca Data Sensor Suhu & Kelembapan (DHT)

Sensor DHT menggunakan protokol komunikasi digital *single-wire*. Artinya, seluruh data (suhu dan kelembapan) ditransmisikan melalui satu kabel data saja. Protokol ini sangat bergantung pada waktu (*timing*). Mikrokontroler harus mengirimkan perintah "minta data" dengan durasi *HIGH* dan *LOW* yang sangat presisi. Setelah itu, sensor akan mengirimkan kembali serangkaian pulsa digital yang panjangnya spesifik menunjukkan nilai suhu dan kelembapan.

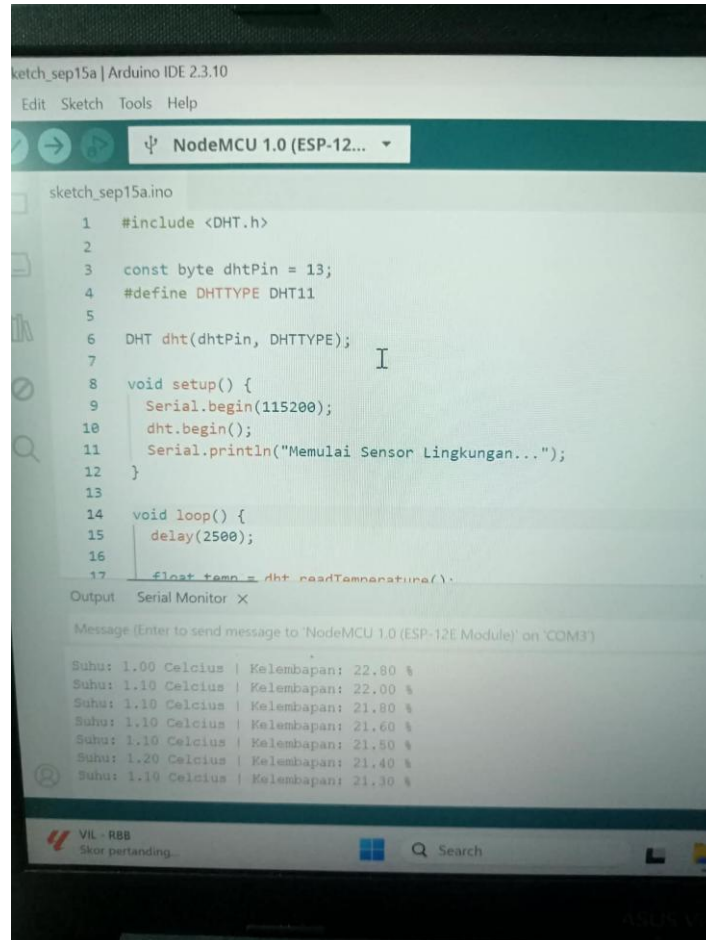


Langkah Kerja Rangkaian:

1. Hubungkan pin **VCC / + / 3V3** pada DHT ke pin **3V3** NodeMCU.
2. Hubungkan pin **GND / -** pada DHT ke pin **GND** NodeMCU.
3. Hubungkan pin **DATA / OUT / S** pada DHT ke pin **D7 (GPIO 13)** pada NodeMCU.

Langkah Kerja Pemrograman:

1. Buat berkas baru (*New Sketch*).
2. Tuliskan kode program berikut:



```
sketch_sep15a | Arduino IDE 2.3.10
Edit Sketch Tools Help
NodeMCU 1.0 (ESP-12...)
sketch_sep15a.ino
1  #include <DHT.h>
2
3  const byte dhtPin = 13;
4  #define DHTTYPE DHT11
5
6  DHT dht(dhtPin, DHTTYPE);
7
8  void setup() {
9    Serial.begin(115200);
10   dht.begin();
11   Serial.println("Memulai Sensor Lingkungan...");
12 }
13
14 void loop() {
15   delay(2500);
16
17   float temp = dht.readTemperature();
18
19   Serial.print("Suhu: ");
20   Serial.print(temp);
21   Serial.print(" Celcius | Kelembapan: ");
22   Serial.print(dht.readHumidity());
23   Serial.println("%");
24 }
```

Output Serial Monitor X

Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM3')

Suhu: 1.00 Celcius	Kelembapan: 22.80 %
Suhu: 1.10 Celcius	Kelembapan: 22.00 %
Suhu: 1.10 Celcius	Kelembapan: 21.80 %
Suhu: 1.10 Celcius	Kelembapan: 21.60 %
Suhu: 1.10 Celcius	Kelembapan: 21.50 %
Suhu: 1.20 Celcius	Kelembapan: 21.40 %
Suhu: 1.10 Celcius	Kelembapan: 21.30 %

3. **Upload** program dan buka **Serial Monitor**. Anda akan melihat data suhu lingkungan dan persentase kelembapan udara dicetak setiap 2.5 detik.

Penjelasan Singkat Kode:

- `#include <DHT.h>`: Mengimpor fungsi-fungsi dari *library* Adafruit DHT yang baru saja Anda instal.
- `#define DHTTYPE DHT11`: Mendefinisikan konstanta preprosesor untuk tipe sensor. (Ubah menjadi DHT22 jika sensor Anda berwarna putih).
- `DHT dht(dhtPin, DHTTYPE);`: Membentuk "objek" pengendali bernama `dht` yang mengingat pin data dan jenis sensor yang digunakan.

- `float temp = dht.readTemperature();`: Perintah ini bertugas membaca suhu dan menyimpannya dalam variabel float (karena mengandung angka desimal).
- `isnan(temp)`: *Is Not a Number*. Fungsi matematika yang bertindak sebagai sistem perlindungan (*error handling*). Jika kabel sensor putus, *library* akan menghasilkan data rusak (Not-A-Number). Blok if ini bertugas mendeteksi kerusakan tersebut dan membatalkan sisa kode di bawahnya dengan perintah `return`;

2.6 Latihan

- **Latihan 1 (Pemahaman Sistem ADC):** Mengapa batas maksimal nilai intensitas cahaya yang terbaca pada Serial Monitor di Praktikum 1 adalah sekitar angka 1023 (bukan 100 atau 1000)? Kaitkan jawaban Anda dengan penjelasan "Resolusi ADC 10-bit".

Angka 1023 adalah batas maksimum dari **Resolusi ADC 10-bit** pada ESP8266 ($2^{10} = 1024$ nilai diskrit, dimulai dari angka 0 hingga 1023).

- **Latihan 2 (Implementasi Algoritma):** Nilai ADC 0-1023 tidak lazim dibaca oleh pengguna akhir (manusia). Modifikasi kode pada Praktikum 1 agar program menghitung dan mencetak persentase intensitas cahaya (rentang 0% hingga 100%). Tuliskan rumus matematika / perintah pembagian yang Anda sematkan di kode tersebut!

Untuk mengubah nilai mentah ADC (0–1023) menjadi format persentase yang mudah dipahami manusia (0%–100%), kita bisa menggunakan rumus perbandingan linier sederhana atau memanfaatkan fungsi bawaan Arduino.

$$\text{Presentase} = (\text{Nilai ADC} / 1023) \times 100$$

```
int ldrValue = analogRead(ldrPin);
float percentage = (float)ldrValue / 1023.0 * 100.0;
```

- **Latihan 3 (Analisis Rangkaian):** Di subbab 2.3.4 disebutkan sifat modul *Relay* yang terisolasi *Optocoupler*. Banyak *Relay* di pasaran yang bekerja secara Active-Low. Jika *Relay* Anda bertipe *Active-Low*, apa yang terjadi pada aktuator (misal: lampu ruangan) yang terhubung ke *Relay* jika program Anda mengeksekusi `digitalWrite(relayPin, HIGH)`?

Pada modul relay bertipe Active-Low:

Logika LOW (0V) akan memicu optocoupler untuk mengaktifkan relay (Kondisi ON / Sakelar tertutup).

Logika HIGH (3.3V/5V) akan memutus arus pemicu sehingga relay nonaktif (Kondisi OFF / Sakelar terbuka).

Oleh karena itu, jika program mengeksekusi `digitalWrite(relayPin, HIGH)`, maka relay akan berada dalam kondisi OFF (mati), sehingga lampu ruangan yang terhubung ke aktuator tersebut tidak akan menyala (padam).

- **Latihan 4 (Pengembangan Logika):** Gabungkan wawasan dari Praktikum 1 Modul 2 ini dengan logika kendali LED dari Praktikum Modul 1. Gunakan blok kondisional if yang berfungsi menyalakan sebuah LED (D1) HANYA JIKA sensor LDR mendeteksi lingkungan yang gelap gulita (misalnya: ambang batas ADC berada di bawah angka 200).

Led D1 akan menyala hanya Ketika nilai ADC LDR di bawah 200. Contoh :

```
If (ldrValue < 200) {  
    digitalWrite(ledPin, HIGH);  
} else {  
    digitalWrite(ledPin, LOW);  
}
```